Sets and Maps

COMP 182 | Lecture 13

Unique elements, key-value pairs,
and choosing a collection

Predict results. Explain the contract. Write the Java.

TestHashSet.java

TestMap.java

Learning goals

Choose a Set or Map for a concrete task.

Predict duplicate handling and iteration order.

Use set operations and map collection views.

Build a hash-backed teaching set and map.

Count words and group values by a key.

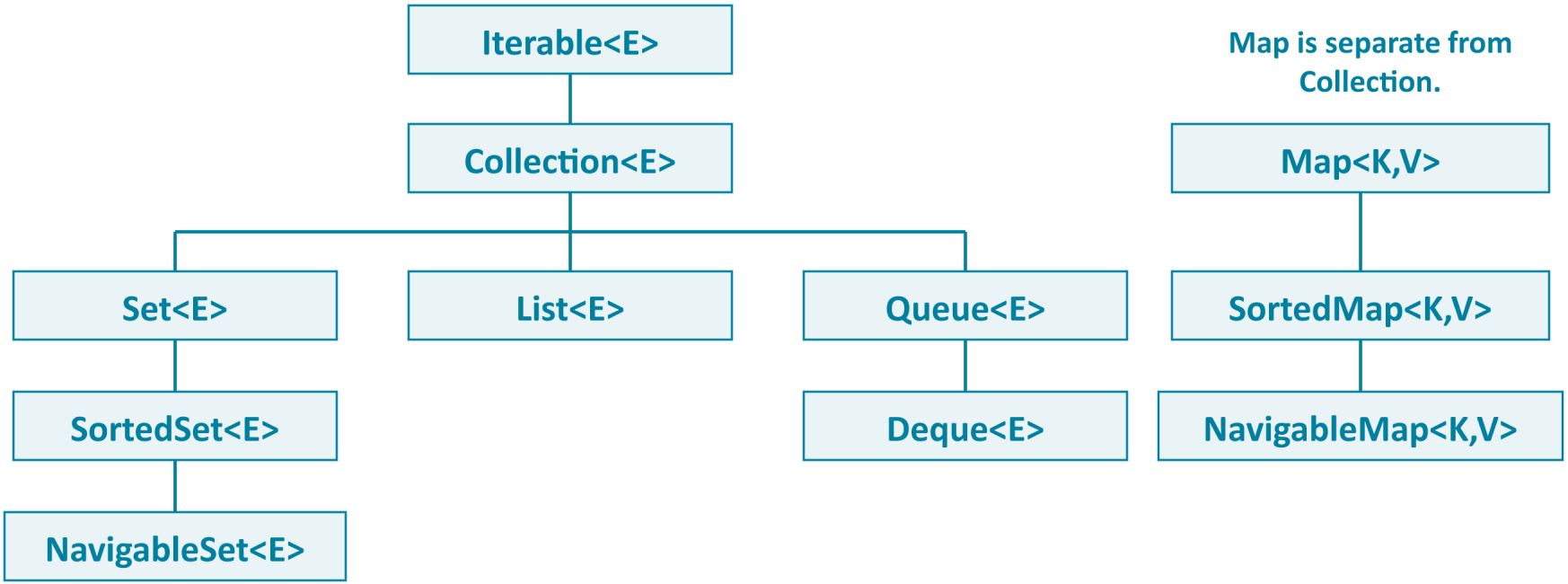
Choose the abstraction

Need	Abstraction	Example
Positions and repeated items	List<E>	Playlist
Unique membership	Set<E>	Enrolled student IDs
Lookup by a key	Map<K,V>	ID to student record
Many values per key	Map<K,Set<V>>	Course to student IDs

Set extends Collection. Map is a separate interface.

Java Collections Framework

Java 17 overview | Lines connect parent interfaces to subinterfaces



COMMON IMPLEMENTATION CLASSES

HashSet LinkedHashSet TreeSet	ArrayList LinkedList	PriorityQueue ArrayDeque LinkedList	HashMap LinkedHashMap TreeMap

TreeSet / TreeMap implement NavigableSet / NavigableMap.
ArrayDeque and LinkedList implement Deque; LinkedList also implements List.

Quiz: read the framework diagram

Select every true statement. Correct each false statement.

A. Map extends Collection.

B. Every Set is also a Collection.

C. LinkedList implements both List and Deque.

D. TreeSet implements NavigableSet.

Which statements describe interfaces, and which name classes?

The Set contract

A set stores distinct elements.

add(e) changes the set only when e is absent.

Set equality depends on members, not order.

Iteration order depends on the implementation.
HashSet, LinkedHashSet, and TreeSet differ.

“No duplicates” does not mean “always unordered.”

TestHashSet.java TestLinkedHashSet.java TestTreeSet.java

Everyday Set operations

Operation	Result or effect
add(e)	true if membership changes
contains(e)	true if an equal member is present
remove(e)	true if a member is removed
size() / isEmpty()	Member count / emptiness
iterator() / for-each	Visit the members

Set<Integer> stores wrapper objects; int values are autoboxed.

TestHashSet.java

Quiz: predict the membership

```
Set<Integer> s = new HashSet<>();
System.out.println(s.add(28));
System.out.println(s.add(78));
System.out.println(s.add(28));
System.out.println(s.size());
```

What prints? Can you predict the iteration order?

SetMembershipQuiz.java

HashSet and an Iterator

```
Set<String> names = new HashSet<>();
names.add("Mia");
names.add("Ada");
names.add("Mia");
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().startsWith("M"))
        it.remove();
}
System.out.println(names); // [Ada]
```

Use the iterator's remove while traversing a mutable set.

TestHashSet.java

Object identity and equality

```
StudentKey a = new StudentKey("S101");
StudentKey b = new StudentKey("S101");
System.out.println(a == b);       // false
System.out.println(a.equals(b));  // true
Set<StudentKey> students = new HashSet<>();
students.add(a);
students.add(b);
System.out.println(students.size()); // 1
```

StudentKey is an immutable record whose ID defines equality.

StudentKey.java

EqualityDemo.java

Hashing rules for elements and keys

Equal objects must have equal hash codes.

Unequal objects may share a hash code.

Do not change equality-relevant fields
while an object is stored as a set member or map key.

Use stable IDs; keep editable details in map values.

StudentKey.java

EqualityDemo.java

Discussion: collisions and mutable keys

Assume equals and hashCode obey their contracts before insertion.

1. Two unequal student IDs have the same hash code. Should a HashSet retain both?

2. Suppose a mutable key's ID changes after insertion. What can happen to contains or remove?

3. Where should editable details such as a student's name or email be stored?

Explain which situation is a normal collision and which breaks an assumption.

LinkedHashSet preserves insertion order

```
Set<String> names = new LinkedHashSet<>();
names.add("Mia");
names.add("Ada");
names.add("Mia");
System.out.println(names); // [Mia, Ada]
names.remove("Mia");
names.add("Mia");
System.out.println(names); // [Ada, Mia]
```

A duplicate add keeps its position. Remove then add moves it.

TestLinkedHashSet.java

TreeSet keeps elements sorted

```
NavigableSet<String> names = new TreeSet<>();
names.addAll(List.of("Mia","Ada","Zoe","Ada"));
System.out.println(names); // [Ada, Mia, Zoe]
System.out.println(names.ceiling("Bea")); // Mia
System.out.println(names.headSet("Mia",false));
// [Ada]
```

TreeSet supports sorted traversal and range queries.

TestTreeSet.java

Discussion: what counts as a duplicate?

```
Set<String> names =
    new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
System.out.println(names.add("Ada")); // true
System.out.println(names.add("ADA")); // false
```

The comparator returns zero. String.equals returns false.

TestTreeSet.java

Quiz: membership and iteration order

```
List<String> input = List.of("Mia", "Ada", "Mia", "Zoe");
Set<String> h = new HashSet<>(input);
Set<String> l = new LinkedHashSet<>(input);
Set<String> t = new TreeSet<>(input);
```

1. Give the size and guaranteed iteration order of h, l and t.

2. After l.remove("Ada"); l.add("Ada");, what is l's order?

Do not infer a HashSet order from one run.

Union, intersection, and difference

Operation	Members	X={54,19,75}; Y={75,12}
Union: X or Y	In either set	{12,19,54,75}
Intersection: X and Y	In both sets	{75}
Difference: X minus Y	In X but not Y	{19,54}
Difference: Y minus X	In Y but not X	{12}

Union and intersection commute. Difference does not.

SetOperations.java

Copy before applying bulk operations

```
Set<Integer> x = Set.of(54,19,75);
Set<Integer> y = Set.of(75,12);

Set<Integer> union = new HashSet<>(x);
union.addAll(y);
Set<Integer> both = new HashSet<>(x);
both.retainAll(y);
Set<Integer> onlyX = new HashSet<>(x);
onlyX.removeAll(y);
```

addAll, retainAll, and removeAll mutate their receiver.

SetOperations.java

Quiz: set algebra and aliasing

1. X={1,2,3}; Y={3,4}. Find X minus Y.

2. What does x.retainAll(y) change?

3. Does Set<Integer> copy = x copy the members?

4. Why can Set.of(1,1) fail?

SetOperations.java

NullAndFactoriesDemo.java

A small teaching Set interface

```
public interface MySet<E> extends Iterable<E> {
    boolean add(E element);
    boolean remove(E element);
    boolean contains(E element);
    int size();
    boolean isEmpty();
    void clear();
}
```

This exercise defines MySet; it is separate from java.util.Set.

MySet.java

MyHashSet.java

A hash-backed set can reuse a map

```
private static final Object PRESENT = new Object();
private final MyHashMap<E,Object> map;

public boolean add(E e) {
    return map.put(e,PRESENT) == null;
}
public boolean contains(E e) {
    return map.containsKey(e);
}
public boolean remove(E e) {
    return map.remove(e) != null;
}
```

Each distinct element becomes one map key.

MyHashSet.java TestMyHashSet.java MyHashMap.java

Case study: count Java keywords

Read source text from a UTF-8 file.

Recognize identifier tokens and test Set membership.

Ignore comments and string/character literals.

Count reserved keywords, not words inside text.

Example: "if" in a string is not an if keyword.

CountKeywords.java

Keyword counting: the membership step

```
String token = source.substring(start,i);
if (KEYWORDS.contains(token))
    result.merge(token,1,Integer::sum);
```

Default sample:
public class Demo { int n; /* while */ }

Expected counts: class=1, int=1, public=1
Total keyword occurrences: 3

The full file handles token boundaries and quoted text.

CountKeywords.java

The Map contract

A map stores key-value pairs.

Each key has at most one associated value.

put with an existing key replaces its value.

Different keys may have equal values.

Map is also called a dictionary or associative array.

Use a stable student ID as a key, not a possibly repeated name.

TestMap.java

Everyday Map operations

Method	Meaning
get(key)	Value, or null for an absent key
put(key,value)	Store mapping; return previous value
remove(key)	Remove mapping; return previous value
containsKey(key)	Distinguish absence from a null value
containsValue(value)	Search for an associated value
size() / clear()	Count mappings / remove all mappings

Null results need care when the map permits null values.

TestMap.java

NullAndFactoriesDemo.java

Trace put and replacement

Statement	Return	Map after statement
put("Mia",88)	null	{Mia=88}
put("Ada",91)	null	{Mia=88, Ada=91}
put("Mia",93)	88	{Mia=93, Ada=91}
size()	2	Two keys, not three insertions

This trace uses an insertion-order LinkedHashMap.

TestMap.java

Quiz: map returns and replacement

```
Map<String,Integer> m = new LinkedHashMap<>();
System.out.println(m.put("A", 10));
System.out.println(m.put("B", 10));
System.out.println(m.put("A", 12));
System.out.println(m.remove("C"));
System.out.println(m.size());
```

1. Write the five output lines.

2. Give the final mappings and their iteration order.

Which operation replaces a value without adding a key?

Choose a map implementation

Implementation	Iteration order	Basic key operations
HashMap	Unspecified	Expected O(1)*
LinkedHashMap	Insertion order by default	Expected O(1)*
TreeMap	Sorted by key	O(log n)

***Suitable hash distribution; resizing affects individual inserts.**

TestMap.java

Convert to a sorted map

```
Map<String,Integer> scores = new LinkedHashMap<>();
scores.put("Mia",88);
scores.put("Ada",91);
scores.put("Mia",93);
System.out.println(scores);
// {Mia=93, Ada=91}
System.out.println(new TreeMap<>(scores));
// {Ada=91, Mia=93}
```

The new map copies mappings. It does not reorder scores.

TestMap.java

Iterate through Map.Entry pairs

```
for (Map.Entry<String,Integer> entry
        : scores.entrySet()) {
    System.out.println(
        entry.getKey() + " -> " + entry.getValue());
}
```

An entry supplies the associated key and value together.

TestMap.java

MapViewsDemo.java

Map collection methods return backed views

Method	Return type	Can values repeat?
keySet()	Set<K>	Keys are unique
values()	Collection<V>	Yes
entrySet()	Set<Map.Entry<K,V>>	One pair per key

On mutable JDK maps, supported removals through a view affect the map.

MapViewsDemo.java

Null values and unmodifiable factories

```
Map<String,Integer> map = new HashMap<>();
map.put("Ada",null);
System.out.println(map.get("Ada")); // null
System.out.println(map.get("Zoe")); // null
System.out.println(map.containsKey("Ada")); // true
System.out.println(map.containsKey("Zoe")); // false
```

Set.of and Map.of create unmodifiable collections and reject nulls.

NullAndFactoriesDemo.java

Quiz: keys, values, and views

1. put("A",4), put("B",4), put("A",7): size?

2. Can values() contain two equal values?

3. Does keySet().remove("A") remove a mapping?

4. Is a HashMap sorted by its keys?

TestMap.java

MapViewsDemo.java

A small teaching Map interface

```
public interface MyMap<K,V> {
    V get(K key);
    V put(K key,V value);
    V remove(K key);
    boolean containsKey(K key);
    boolean containsValue(V value);
    int size();
    boolean isEmpty();
    void clear();
    // Snapshot collection methods are in the full file.
}
```

MyMap is a separate exercise API with non-null keys and values.

MyMap.java

MyHashMap.java

Separate chaining: put an entry

Hash the key to a bucket.

Search that bucket using key equality.

If present: replace the value and return the old value.

If absent: add one entry and increase size.

Grow and rehash when the load threshold is exceeded.

Equal keys share a bucket; unequal keys can collide there.

MyHashMap.java

TestMyHashMap.java

Teaching snapshots and JDK views differ

```
MyMap<String,Integer> scores = new MyHashMap<>();
scores.put("Ada",95);
scores.put("Mia",95);
Set<String> keys = scores.keySet();
scores.remove("Ada");
System.out.println(keys.contains("Ada")); // true
System.out.println(scores.size()); // 1
```

Our teaching methods return unmodifiable independent snapshots.

TestMyHashMap.java

MyHashMap.java

Discussion: views, copies, snapshots

```
Map<String,Integer> live = new HashMap<>();
live.put("A", 1);
Set<String> view = live.keySet();
Set<String> copy = new HashSet<>(view);
live.put("B", 2);
view.remove("A");
```

1. What are live, view and copy at the end?

2. Does copy.add("C") change live?

3. How does our teaching map's keySet snapshot differ from copy?

Case study: count word occurrences

Normalize case using Locale.ROOT.

Split text into letter/digit runs; ignore empty tokens.

Use each word as the key and its count as the value.

Increment the count with merge.

Copy to a TreeMap for sorted output.

Define tokenization before comparing counts.

CountOccurrenceOfWords.java

Word counting with merge

```
Map<String,Integer> counts = new HashMap<>();
for (String word : text.toLowerCase(Locale.ROOT)
        .split("[^\\p{L}\\p{N}]+")) {
    if (!word.isEmpty())
        counts.merge(word,1,Integer::sum);
}
return new TreeMap<>(counts);
```

Java, java! Maps help; maps count. → java=2, maps=2

CountOccurrenceOfWords.java

Multisets and multimaps

Structure	Representation	Example
Set	Set<String>	{Ada, Mia}
Multiset / bag	Map<String,Integer>	Ada occurs 3 times
Map	Map<String,Integer>	Ada has score 95
Multimap	Map<String,Set<String>>	COMP182 has Ada and Mia

A map still has unique keys when its values are collections.

CountOccurrenceOfWords.java

MultimapDemo.java

Group values with computeIfAbsent

```
Map<String,Set<String>> courses = new TreeMap<>();
courses.computeIfAbsent("COMP182",
    k -> new TreeSet<>()).add("Ada");
courses.computeIfAbsent("COMP182",
    k -> new TreeSet<>()).add("Mia");
courses.computeIfAbsent("COMP182",
    k -> new TreeSet<>()).add("Ada");
System.out.println(courses);
// {COMP182=[Ada, Mia]}
```

Create the group once, then add members to that group.

MultimapDemo.java

Quiz: grouped values and duplicates

```
Map<String,Set<String>> groups = new TreeMap<>();
for (String name : List.of("Mia", "Ada", "Mia"))
   groups.computeIfAbsent("COMP182",
      k -> new TreeSet<>()).add(name);
groups.computeIfAbsent("COMP110",
   k -> new TreeSet<>()).add("Mia");
```

1. What are groups.size() and the COMP182 group's size?

2. What does printing groups show?

3. What changes if each group uses an ArrayList instead of a TreeSet?

Discussion: select the collection

1. Remove duplicate IDs, keeping first-seen order.
2. Find a student record by ID.
3. Print unique names in sorted order.
4. Record all students for each course.
5. Count how often each word occurs.

Exit quiz

1. Why must equals and hashCode agree?

2. Why does a repeated map key not increase size?

3. How do backed views differ from snapshots?

4. When would a TreeMap be worth O(log n)?

5. What changes if a multimap uses List values?

Practice after class

Extend word counting to print the top three words.
Use a documented rule for tied counts.

Build a course enrollment multimap using student IDs.

Compare JDK map views with the teaching snapshots.

Trace a collision and a resize in MyHashMap.

CountOccurrenceOfWords.java MultimapDemo.java MyHashMap.java